TECHNICAL ARCHITECTURE & TASK ROADMAP

Smart Fuel Management System — Redis Caching Implementation Plan

A rigorous, function-by-function specification roadmap for introducing distributed caching across Java Spring Boot microservices while preserving operational safety, financial integrity, and real-time safeguards.

Project: Smart Fuel Management System

Target: Microservices Architecture

Document Type: Implementation Task Specification

Status: Approved for Execution

Version: 1.0.0

Scope: 40 Evaluated Functions

1. Architectural Rules & Caching Principles

This implementation roadmap dictates how Redis caching must be introduced across the Smart Fuel Management System. All tasks must follow Spring Boot's caching abstraction (`@Cacheable` , `@CacheEvict` , `@Caching`) with strict adherence to data governance boundaries.

> **Core Directive:** This specification is a task plan, not an application code modification. No source code, configuration files, `pom.xml` , or database schemas may be altered during documentation review.

Key Architectural Principles

- **Read-Operation Focus:** Caching must be placed around appropriate read operations, never around controllers indiscriminately.
- **Static & Historical Data:** Prioritize caching stable metadata, frequently requested display data, and read-heavy historical summaries.
- **Fixed TTL Strategy:** Use fixed TTLs by default rather than resetting TTL on every cache hit to prevent indefinite cache pollution.
- **Eviction Integrity:** Invalidate affected cache entries immediately after successful mutations. Cross-service boundaries require explicit event-driven invalidation.
- **Absolute Safety Exclusion:** Never treat cached values as authoritative for security decisions, payment execution, fuel-session state, or operational safety.

2. Implementation Phases

Phase	Focus Area	Objective & Deliverables
Phase 1	Preparation & Existing Review	Inspect Redis connection topology, serialization standards, and confirm isolation of 60-min pending verification and 45-sec pump heartbeat mechanisms.
Phase 2	P1 High-Priority Caching	Implement approved high-priority read caching for tank specs, pricing summaries, vehicle batches, and transaction dashboard aggregates with strict invalidation.
Phase 3	P2 Optional Caching	Implement lower-priority display and list caching (station prices, vehicle counts, display names) after P1 acceptance testing passes.
Phase 4	Verification & Performance	Execute comprehensive testing suite validating cache hits, TTL expiration, invalidation correctness, and Redis failure resilience.

3. Existing Redis Mechanisms to Preserve

Specialized Redis behaviors currently operating within the microservices must remain completely isolated from ordinary application response caching:

- **AUTH-SERVICE Verification:** Stores pending user verification data with a 60-minute expiration. This is temporary security workflow state, not an ordinary cache. Its expiration, cleanup workflow, OTP separation, and failure behavior must remain untouched.
- **Pump Heartbeat Mechanism:** Uses the `pump:heartbeat:` prefix with a 45-second TTL. Heartbeat writes, refreshes, and connection status detection must continue to operate without interference or false ONLINE/OFFLINE status changes.

4. Detailed Function Checklist: P1 High-Priority Candidates

The following P1 functions are approved for high-priority caching due to high read frequency and clear invalidation boundaries.

Function & Class	Priority	Caching Strategy & Data	TTL & Invalidation Rules
`VehicleSpecService.getTankCapacity(...)`	P1	Cache known specification results. Preserve existing behavior for missing specs.	TTL: 24h. Invalidate on spec update events.
`FuelPriceServiceImpl.getPrice(...)`	P1	Conditional short-TTL caching for current active fuel pricing.	TTL: 5m. Invalidate on price modification write.
`VehicleServiceImpl.getVehicleSummaryDetails(...)`	P1	Short-TTL transaction-enrichment cache for vehicle summaries.	TTL: 10m. Invalidate on vehicle profile updates.
`VehicleServiceImpl.getVehiclesByIds(...)`	P1	Batch metadata caching. Key must explicitly reflect requested IDs list.	TTL: 15m. Invalidate on individual vehicle mutation.
`UserServiceImpl.getUserForPayment(...)`	P1	Cache non-authoritative profile display data only. Never bypass auth.	TTL: 5m. Invalidate on profile update.
`UserServiceImpl.getUsersByIds(...)`	P1	Batch display-metadata caching with parameter-aware key generation.	TTL: 10m. Invalidate on user batch updates.

Detailed Function Checklist: P1 Candidates (Continued)

Function & Class	Priority	Caching Strategy & Data	TTL & Invalidation Rules
`StationServiceImpl.getStationsByIds(...)`	P1	Batch static-metadata caching for fueling stations.	TTL: 1h. Invalidate on station configuration changes.
`StationServiceImpl.getDetailsForTransaction(...)`	P1	Short-TTL historical display cache where safe for transactions.	TTL: 15m. Invalidate on station metadata updates.
`TransactionServiceImpl.getByUserId(...)`	P1	Short-TTL historical user transaction summary cache.	TTL: 2m. Invalidate on new transaction creation.
`TransactionServiceImpl.recentTransactions(...)`	P1	Very short-TTL recent-history cache for dashboard widgets.	TTL: 60s. Invalidate on transaction completion.
`TransactionServiceImpl.getDashboardSummary(...)`	P1	Short-TTL aggregate financial summary cache.	TTL: 5m. Invalidate on transaction settlement.
`TransactionServiceImpl.getStatistics(...)`	P1	Parameterized short-TTL aggregate caching reflecting filters and dates.	TTL: 10m. Invalidate on period close or batch insert.

5. Detailed Function Checklist: P2 Optional Candidates

Implemented only after P1 acceptance criteria are fully satisfied.

Function & Class	Priority	Caching Strategy & Data	TTL & Invalidation Rules
`FuelPriceServiceImpl.getStationPrices(...)`	P2	Cache display reads for station pricing. TTL: 10m.	Invalidate on price update.
`FuelPriceServiceImpl.getStationHistoryPrices(...)`	P2	Optional short-TTL historical price caching. TTL: 30m.	Invalidate on historical correction.
`VehicleServiceImpl.findAllByUserId(...)`	P2	Short-TTL user vehicle-list cache. TTL: 5m.	Invalidate on vehicle add/remove.
`VehicleServiceImpl.getVehicleCount(...)`	P2	Short-TTL dashboard-count cache. TTL: 5m.	Invalidate on vehicle count change.

6. Functions That Must NOT Be Cached

The following security, financial, verification, and operational functions must never be placed in ordinary application caches due to critical safety and transactional requirements:

Function Class & Methods	Safety Classification	Mandatory Restriction Rationale
`PendingUserServiceImpl.findByEmail / findByPhone`	**DO NOT CACHE**	Temporary security workflow state (60-min verification), not ordinary cache.
`OtpStorageServiceImpl.getEmailOtp / getPhoneOtp`	**DO NOT CACHE**	One-time expiring security state; caching compromises auth integrity.
`PendingIdentifierChangeServiceImpl.findById`	**DO NOT CACHE**	Preserves temporary identifier-change semantics without stale reads.
`JWTGenerator.validateToken`	**DO NOT CACHE**	Authorization decisions must never bypass expiration, revocation, or security checks.
`PaymentServiceImpl.{preAuth, capturePayment, handlePaymentEvent, reconcile}`	**DO NOT CACHE**	Financial side effects and payment-state transitions require absolute ACID accuracy.
`FuelSessionServiceImpl.{startSession, stopSession, pauseSession, resumeSession}`	**DO NOT CACHE**	Active operational fueling workflows require real-time database authority.
`TransactionServiceImpl.createFromPayment`	**DO NOT CACHE**	Preserves durable, idempotent transaction creation workflows.
`PumpHeartbeatServiceImpl.{recordHeartbeat, getConnectionStatus, ...}`	**DO NOT CACHE**	Maintains real-time pump heartbeat semantics and dedicated 45-second TTL.
`VehicleServiceImpl.resolve(...) for fueling`	**DO NOT CACHE**	Vehicle authorization and operational status decisions must be live.
`StationServiceImpl.validateStation(...) for operations`	**DO NOT CACHE**	Operational eligibility decisions require real-time validation checks.

> **Strict Safety Mandate:** Never confuse temporary verification storage or pump heartbeats with standard application caching. Bypassing these restrictions introduces severe security and financial risks.

7. Verification, Testing & Acceptance Criteria

Prior to production deployment, all caching implementations must pass comprehensive automated test suites and verify the following acceptance criteria:

- **Review Gate:** Every approved cache candidate is implemented only after its specific task is reviewed.
- **Key Uniqueness:** Cache keys uniquely identify relevant data, filters, pagination, and parameters without collision.
- **TTL Justification:** TTLs are explicitly configured, documented, and justified based on data volatility.
- **Mutation Invalidation:** Cache invalidation executes reliably after successful relevant updates across microservices.
- **Resilience:** Redis unavailability or serialization failures do not corrupt application behavior or cause unhandled exceptions (fallback to DB).
- **Mechanism Preservation:** Verification respects its 60-minute expiration; heartbeat respects its 45-second TTL.
- **Performance Proof:** Performance improvements are empirically measured via query count reductions rather than assumed.

8. Final Compliance & Sign-Off

Upon successful execution of all implementation phases and verification tests, the caching roadmap is marked complete. No application files, configuration files, or build scripts were modified during the generation of this technical planning document.

> **Document Status:** Ready for architectural review and sprint task assignment. Output saved as `Smart_Fuel_Redis_Caching_Implementation_Plan.pdf`.